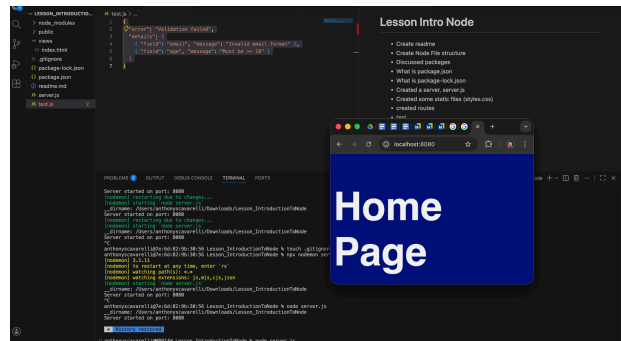


A1: Multi-page Web Application (10%, Group)

OVERVIEW

In this assignment, you will work with your group to design and develop a medium-fidelity web application using static web assets, multiple web pages and associated routes, and a basic REST API interaction between a client and server (no database this time - simulated with a JSON object saved on the server).



Note that the application type is up to you. As long as it meets the requirements below, you are encouraged to be creative with the kind of application your team develops (e.g., a video game, SaaS app, support tool, drawing tool, or puzzle).

Requirements:

Please complete the following requirements:

- **Develop** (1) Node.js server using Express (called `server.js`).
 - **Add** (1) JSON object (either as a variable or `.json` file) that contains a list of items that you will add and remove.
- **Develop** (3) different HTML web pages. One or more should allow the user to send GET, POST, and DELETE requests to the server.
- **Develop** (1) REST API that supports (3) functions with appropriate status codes used:
 - GET request from a client page to receive a list.
 - POST request from a client page, requesting the server to add an item.
 - DELETE request from a client page, asking the server to delete an item.
- **Record** a short video of your working project (≤ 60 s, .MP4 format).
- **Write** a reflection in the form of a ***README.md*** included with the project with the following components:

- **Overview:** Provide a brief description of the web application's concept and purpose, and how it could be extended in the future.
- **Documentation:** Describe how to run the project and how to use it.
- **Reflection:** Provide a brief overview of the submitted content, and any challenges and/or successes you had.

Recommended:

- Consider developing an interface you can extend in future assignments (e.g., adding a database or React).
- Consider how to make your design modular.
- Using Git to save history and collaborate more effectively with your peers.
- Be creative with your concept; e.g., your web application could be a game using Three.js or a similar library to add and remove objects, rather than a simple HTML list.
- Handle invalid requests eloquently. E.g., 404.

SUBMISSION

One member from each group will submit a “**groupName_CPS630_A1.zip**” folder containing all requirements (code - the entire node project, including package.json, video, and written reflection - **do not** include *node_modules*) to Brightspace by the specified deadline. NOTE: *Class policies on Brightspace define the late and plagiarism policies.*

LEARNING OUTCOMES

- To practise developing web applications
- To understand the web application technology stack and the client-server mode.
- To develop a method of communication between clients and a server.
- To reflect on developing web applications.

EVALUATION RUBRIC

	Excellent	Good	Satisfactory	Needs Improvement
Node/ Express Setup (15)	package.json correct; Express installed; server.js cleanly starts; no errors.	Server runs with minor config issues.	Server runs, but with warnings or awkward setup.	Server fails to start or major runtime errors.
Static Assets & Layout (15)	Static CSS and images served correctly; layout looks coherent across pages; Express static middleware used appropriately.	Static files work with minor path or styling issues; layout is mostly consistent.	Static assets are present but poorly organized or inconsistently used.	No meaningful styling; static assets misconfigured or missing.
HTML Routing & Multi-page Structure (20)	All required HTML routes (e.g., /, /about, /contact, /items) work; each returns meaningful, distinct content with shared layout and navigation. Errors from invalid requests are handled.	Most routes are present and functional; layout is mostly consistent. Some errors are not transparently handled.	Routes are mostly present, but some links are broken, or the layout is inconsistent. Invalid requests are not handled.	Few routes implemented; navigation broken.
REST API (20)	Returns correct JSON list and single item; uses appropriate status codes (e.g., 200, 404); Content-Type set; handles errors.	API mostly works; minor issues with error handling or headers.	Basic JSON returned, but status codes or error handling are inconsistent.	API not implemented or largely incorrect.
Code Organization & Quality (10)	Clean modular structure (routes, data, views); readable code; clear comments.	Reasonable structure; working, but could be clearer.	Most code in one or two files; hard to navigate; still workable.	Highly disorganized; difficult to follow; poor style.
Documentation and Reflection (10)	README clearly documents setup, routes, and API; multiple meaningful commits.	The README presents all required written components. Mostly clear.	README incomplete. Some written components are missing.	Little/no/unclear documentation.
Overall Polish & Professionalism (10)	App feels coherent; no obvious console errors; meaningful content; small touches (titles, favicon), Git use, improve experience.	Generally usable; minor rough edges or warnings. Above average compared to peers.	Functional but rough; little attention to user experience.	Barely functional; clearly rushed.